

Teaching Exam

Subject – Computer Science

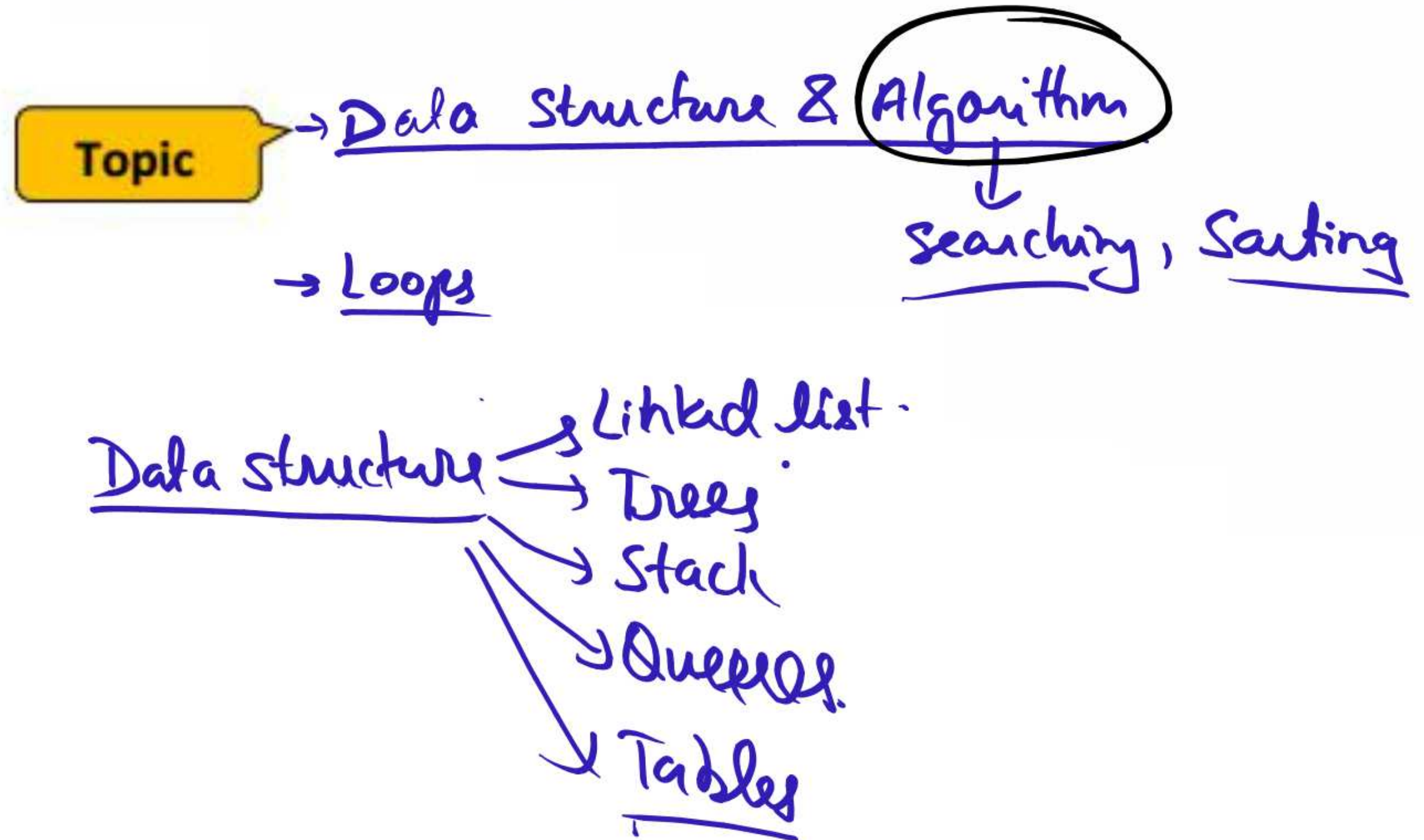
**Data Structure and
Algorithm**



Lecture No –1

By–Satyendra Chaudhary Sir

Topics to be Covered



What is a loop in Python?

A loop in Python is a control flow statement that allows you to execute a piece of code repeatedly until a specific condition is met.

Loops in Python are necessary for operations that require repetitive execution,
such as iterating through a Python list of items or doing calculations many times.

Different Types of Loops in Python

To understand the Loop Structures in Python, there are 3 Types of Loops in Python:

while loop ✓
for loop ✓
nested loop

while (True) ~~False~~
{
==
while :
{ while } }

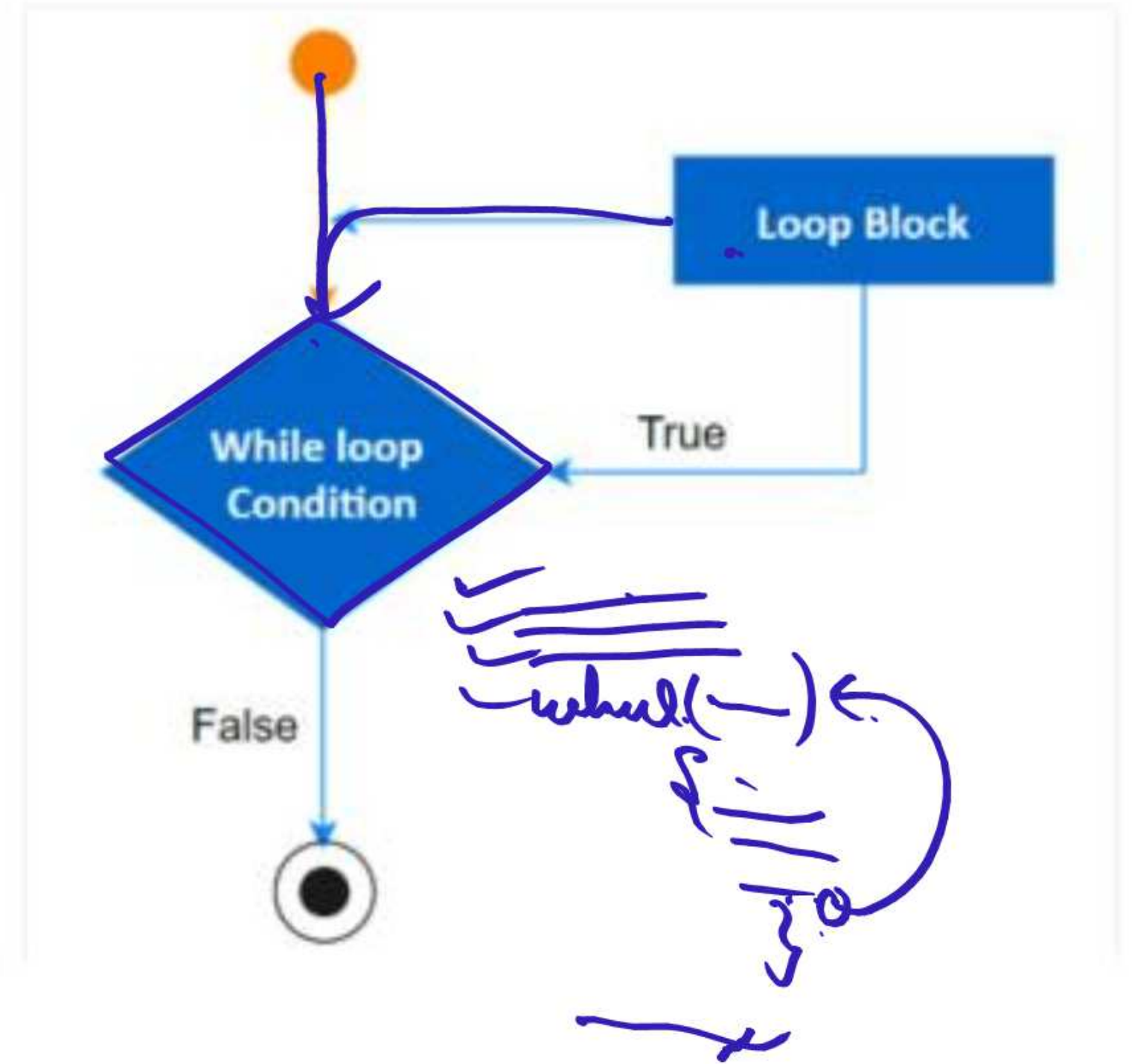
for ({
}

Python while loop

A while loop in Python is used to repeat a block of code as long as a certain condition is true.

It keeps running the loop body again and again until the condition becomes false.

This is useful when you don't know in advance how many times the loop should run.




```
count = 0
while (count < 10):
    print ('The count is:', count)
    count = count + 1
print ("Good bye!")
```

A while loop is a control flow statement that allows you to repeatedly run a block of code as long as a given condition is true.

$list = [2, 5, 4]$

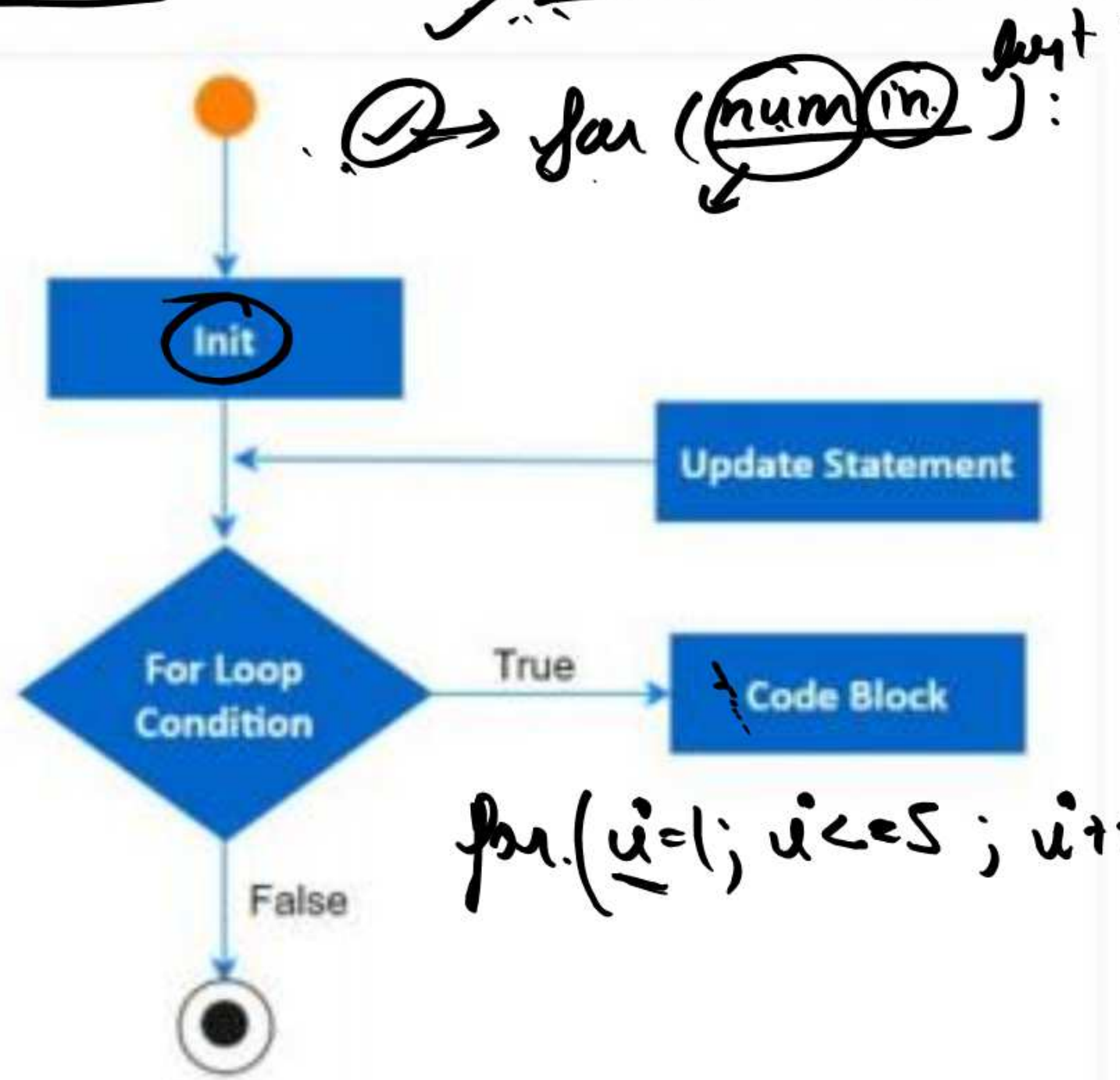
✓ —
✓ —

Python for loop

A for loop in Python is used to go through a sequence of items, like a list, a string, or a range of numbers.

It repeats the block of code once for each item in the sequence.

This makes it very useful when you already know how many times you want to loop.



```
for letter in 'Python': # First Example
    print('Current Letter:', letter)
fruits = ['guava', 'apple', 'mango']
for fruit in fruits: # Second Example
    print('Current fruit:', fruit)
print("Good bye!")
```

P
y
t
h
o
n

guava
apple
mango

Python Nested loop

A nested loop in Python means using one loop inside another loop.

It is commonly used when you need to repeat actions in a multi-level structure, like printing patterns or working with multi-dimensional lists (like 2D arrays).

```
i = 2
while(i < 100):
    j = 2
    while(j <= (i/j)):
        if not(i%j): break
        j = j + 1
    if (j > i/j) : print (i, " is prime")
    i = i + 1
print ("Good bye!")
```

Loop Control Statements in Python

In Python, loop control statements are used to alter the regular flow of a loop.

They enable you to skip iterations, end the loop early, or do nothing at all.

Python's main three loop control statements are:

- 1-Break
- 2-Continue
- 3-Pass

✓
✓
✓
✓
for(, i=4
"continue"
✓

Break

The break statement is used to quickly stop the loop regardless of whether the loop condition is still true.

This signifies that the loop will be terminated and the next statement outside of the loop will be executed.

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

0-4
5

This code in the Python Online Compiler prints the numbers 0 to 4, but then it exits the loop and moves on to the next statement.

Continue

The continue statement is used to skip the current loop iteration and go to the next iteration.

This implies that the code from the current iteration will not be performed, and the loop will continue to iterate until the loop condition is satisfied.

$$i \% 2 == 0$$

```
for i in range(10):  
    if  $i \% 2 == 0$ :  
        continue  
    print(i)
```

0 ✓
1 ✓
2 ✓
3 ✓
4 ✓
5 ✓
6 ✓
7 ✓
8 ✓
9 ✓

This code generates the odd numbers 1 through 9.
Any iterations where the integer is even are skipped
by the continue statement.

$$\frac{0}{2} = 0 \quad \text{Rem} = 0$$

`break` vs `continue` in Python Loops

Feature	<code>break</code>	<code>continue</code>
Purpose	<u>Exits the loop entirely</u>	Skips the current iteration only
Usage	<u>When you want to stop the loop early</u>	When you want to skip certain steps
Affects	<u>Ends the loop and moves to next statement</u>	Moves to the next loop iteration

Pass

The pass statement is a null statement that has no effect. It can be utilized as a placeholder in control statements or empty loops.

```
for i in range(10):  
    # Do nothing  
    pass
```



The diagram illustrates the range function in the code. A handwritten arrow points from the range(10) in the code to a handwritten tuple (0, 10, 1). Below the tuple, the numbers 0, 1, 2, 3, 4, 5 are written vertically, representing the sequence of values generated by the range function.

This code will not print anything because the pass statement does nothing. It is frequently used as a placeholder in control statements or empty loops.

for loop

- ✓ Use when you know in advance how many times you want to iterate (e.g., over a list, range, string, etc.).

while loop

- ✓ Use when you don't know in advance how many times to iterate and want to loop until a condition is false.

Print Even Numbers from 1 to 20

for i in range(2, 21, 2):
print(i)

2
4
6
8

(2, 21,
2
3

20

$s[0, 7, -]$

for ($i=2$, $i < 21$, $i++2$)

for s in 'Explores':

Sum of Digits of a Number

num = 1234

sum_digits = 0

while num > 0:

digit = num % 10

sum_digits += digit

num // = 10

num = 123

digit = 4

sum_digits = 4

digit = 3

sum = 7

$$\frac{123}{10} \Rightarrow \underline{12.3} \Rightarrow \underline{12}$$

$$123 / 11 \Rightarrow 11 \dots$$

$$123 // 11 \Rightarrow 11 \dots \Rightarrow \underline{11}$$

Check if a Number is Prime

```
num = 17
is_prime = True
```

```
if num < 2:
    is_prime = False
else:
```

```
    for i in range(2, int(num**0.5)+1):
        if num % i == 0:
            is_prime = False
            break
```

```
print(f"{num} is Prime? {is_prime}")
```

$$17^2$$

$$17 \% 2$$

$$17 \% 3$$

$$17 \% 4$$

$$17 \% 5$$

$$\frac{0.8}{1}$$

$$(17)^{\frac{1}{2}} = 4.1$$

$$\underline{17 \% 2} =$$

$$\% 3 =$$

$$\underline{\% 4 =}$$

DSA

Data Structure & Algorithm

⇒ Complexity ⇒

→ Time complexity ⇒

→ Space complexity ⇒

→ Just by algorithm or pseudocode we can analyse.

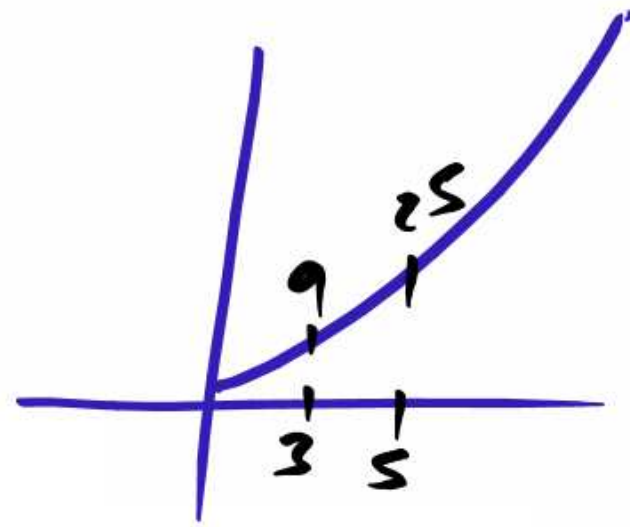
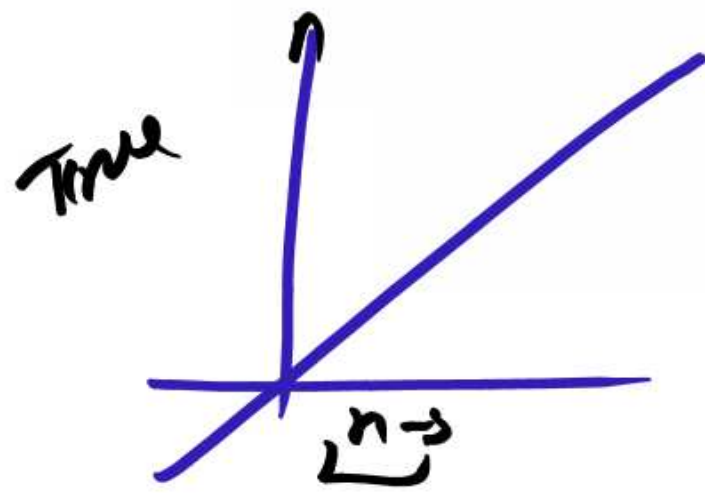
Complexity:

* The mathematical tool used to analyse and describe time complexity is Asymptotic notations.

Big(O) notation- (worst case.)

Ω (omega) notation (best case.)

theta notation (average case.)



→ 1^2 → 2^3

$$1 + 2 + 3 + 4 + 5 + 6 + \dots + n = \frac{(n)(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + 4^2 + \dots = \frac{n(n+1)(2n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots = \left(\frac{n(n+1)}{2} \right)^2$$



2 mins Summary

Topic

Data Structure and Algorithm

THANK YOU